# Profiling Vision-Language Models

## Performance, energy, and where the time actually goes

Vladislav Smirnov

*github.com/smirnovlad/vlm-profiler*

April 2026

# Overview


1. Goal and setup
2. Baseline results
3. What didn't work
4. Where the time goes
5. Insights and next steps

# The question

**Project goal:** understand how VLMs scale and where to spend optimization effort.

Two concrete questions:

1. Which models are Pareto-optimal on quality / latency / energy?
2. Which parts of the model dominate cost?

**Approach:** vary one axis at a time, measure four things directly, then split the dominant model into sub-components.

# Setup

## 7 models, 3 datasets, 5 axes, 4 metrics

**Models (7)**

- `blip2-opt-2.7b`, `blip2-flan-t5-xl`
- `instructblip-flan-t5-xl`
- `instructblip-vicuna-7b`
- `llava-1.5-7b`, `llava-1.5-13b`
- `fuyu-8b`

**Datasets** (300 samples each):
ScienceQA, TextVQA, COCO Captions

**Axes varied**

- Resolution: 224 / 336 / 448 / 512
- Prompt: 10 / 50 / 100 / 200 tokens
- Batch: 1 / 2 / 4 / 8
- FP32, FP16, `torch.compile`, FA2
- GPU vs. CPU

**Metrics**: latency, FLOPs, energy (J), WER.
**Hardware**: 2× RTX 5880 Ada, 48 GiB.

# What the data looks like

## One sample from each dataset

**ScienceQA**

Q: Which animal's mouth is also adapted for bottom feeding?

A: *armored catfish*

**TextVQA**

Q: What number is on the player's jersey?

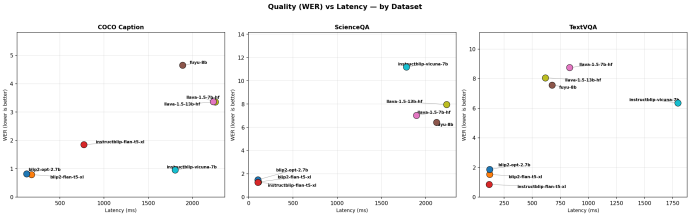
A: *22*

**COCO Captions**

Task: describe the image.

Ref: *A bicycle replica with a clock as the front wheel.*

# Baseline: two clear tiers




| Model | Lat ms | Energy J |
|---|---|---|
| blip2-flan-t5-xl | 136 | 31 |
| blip2-opt-2.7b | 115 | 29 |
| instructblip-t5-xl | 330 | 70 |
| fuyu-8b | 1 562 | 404 |
| llava-1.5-7b | 1 654 | 450 |
| llava-1.5-13b | 1 704 | 470 |
| instructblip-vicuna-7b | 1 793 | 487 |

**Pareto winner**

`blip2-flan-t5-xl`
WER 1.21, 136 ms, 31 J

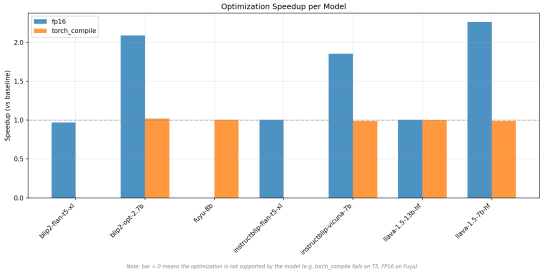
# Other axes: short version

**Resolution 224 → 512:** flat everywhere except Fuyu ($+15\%$ at 512 — raw image patches as text tokens).

**Prompt 10 → 200:** T5 slows $2{-}3.5\times$; 7B+ decoder-only barely move.

**Batch 1 → 8:** small models gain throughput near-linearly; 7B+ scale linearly in *latency* — no throughput gain.

**CPU vs GPU:** 15–30× gap. Impractical for 7B+.

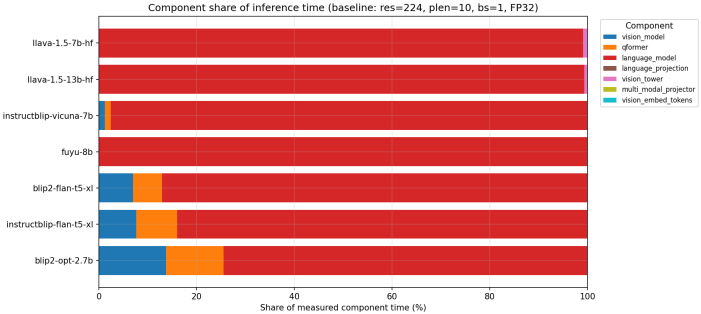



FP16 helps OPT/Vicuna/LLaMA (1.85–2.2×). *Slows* T5. `torch.compile`: no measurable speedup.

# What didn't work, and why

| Attempted | Why it failed / what we did |
|---|---|
| `idefics2-8b` | Pixel-values shape mismatch on `transformers` 5.x ("expected 5, got 4"). Dropped from results. |
| `cogagent`, `moondream2`, `llava-400m-fp16` | Bespoke `trust_remote_code`; last one isn't on HF (likely typo in spec). Used `llava-1.5-7b` for same-family comparison. |
| `torch.compile` on T5 | Compiler bails on encoder-decoder graph. No measurable speedup on the others either. |
| FP16 on T5 | *Slower* than FP32 — internal dtype casts in T5's forward. |
| FP16 on `fuyu-8b` | Float / Half dtype mismatch in patch embedding. |
| `flash-attn` pip | Build needs `nvcc` $\geq$ 11.7; system has 11.5. Used PyTorch SDPA, which dispatches FA2 in FP16 on Ada. |
| Batched `fuyu-8b` | Processor returns lists for `batch>1`. Skipped batch sweep for Fuyu. |

# The punchline: where the time goes



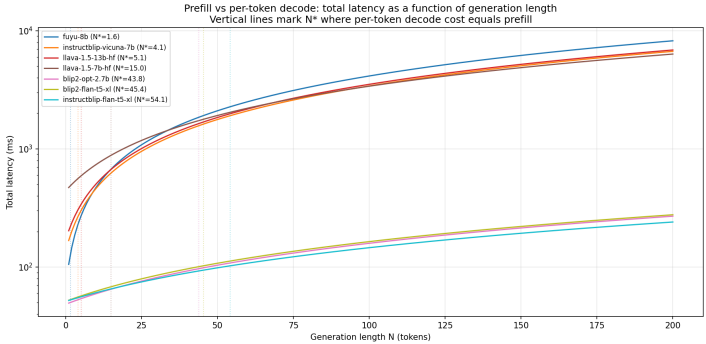

**LLM decoder share**

- BLIP-2 OPT 2.7b: **74%**
- BLIP-2 / Instruct-T5: 84–87%
- 7B+ models: **98–99.99%**

Forward hooks + CUDA events on top-level submodules.

**Optimize the decoder.**

# Inside the decoder: prefill vs. per-token decode




$t(N) = t_{\text{prefill}} + N\, t_{\text{decode}}$
fit at $N$=$1, 50$.

**Crossover $N^*$:**

- fuyu-8b: 1.6
- vicuna-7b: 4.1
- llava-13b: 5.1
- llava-7b: 15
- BLIP-2: 44–54

Short → prefill (FA2). Long → decode (spec., KV quant).

# FlashAttention-2: smaller win than expected

| | SDPA prefill | eager prefill | FA2 speedup | decode/tok (SDPA / eager) |
|---|---|---|---|---|
| `llava-1.5-7b` | 94.5 ms | 105.4 ms | **1.11×** | 15.3 / 15.6 ms |
| `llava-1.5-13b` | 169.0 ms | 181.3 ms | **1.07×** | 33.0 / 34.2 ms |

## Why so modest?

At plen=10 the LLaVA input is $\approx 586$ tokens (10 text + 576 image patch). The eager $O(N^2)$ attention matrix still fits in cache at $N \approx 600$.

## The real lever was the dtype flip

FP32→FP16 on `llava-7b`: 4.6× prefill, 1.9× decode. FA2 inside FP16 adds 1.07–1.11×. For short-answer VQA, FA2 is not the lever — but it would scale at long context.

# Practical insights

1. **For deployment today:** `blip2-flan-t5-xl` or `blip2-opt-2.7b`. The 7B+ tier's quality gain does not justify $\sim$15$\times$ cost.
2. **Optimize the decoder, not the vision tower**, on every model $\geq$ 7B.
3. **Match the optimization to generation length** via $N^*$: short answers $\rightarrow$ prefill-side (FA2, chunked prefill); long answers $\rightarrow$ decode-side (speculative, INT8/INT4, KV-cache quant).
4. **FP16 first, attention kernels second.** Dtype flip is a 2–4$\times$ lever; FA2 inside FP16 is 1.07–1.11$\times$ at short context.
5. **T5 is a trap with current tooling**: `torch.compile` fails, FP16 makes it slower.
6. **Fuyu's design choice has a cost**: tokenized image patches inflate KV cache $\rightarrow$ 41 ms/token despite being 8B.

# Limitations and what's next

**Limitations**

- FLOPs for T5 backbones estimated as $2 \times$ params; calflops doesn't trace cleanly.
- Energy from `nvidia-smi` at 100 ms polling — a few percent precision, not lab-grade.
- No quantization (INT8/INT4) measured.
- FA2 evaluated only at short context; benefit at $N \gg 1k$ not captured.

**Next steps**

- INT8 / GPTQ / AWQ on 7B+ tier (decode-bound, biggest win).
- Speculative decoding with a small draft model.
- Long-context FA2 sweep (resolution 1024+, multi-image prompts).
- Fix the `idefics2-8b` integration.

# Thanks

## Questions?

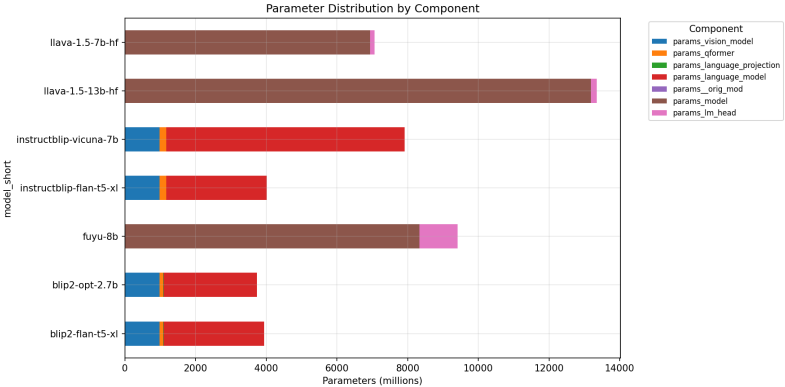
Repo: `github.com/smirnovlad/vlm-profiler`
Report: `report.pdf`

# Backup: per-dataset WER

| Model | COCO Caption | ScienceQA | TextVQA |
|---|---|---|---|
| blip2-flan-t5-xl | 0.79 | 1.30 | 1.54 |
| blip2-opt-2.7b | 0.82 | 1.49 | 1.87 |
| instructblip-flan-t5-xl | 1.85 | 1.28 | **0.86** |
| instructblip-vicuna-7b | **0.96** | 11.18 | 6.36 |
| fuyu-8b | 4.65 | 6.42 | 7.57 |
| llava-1.5-7b-hf | 3.37 | 7.02 | 8.75 |
| llava-1.5-13b-hf | 3.36 | 7.96 | 8.06 |

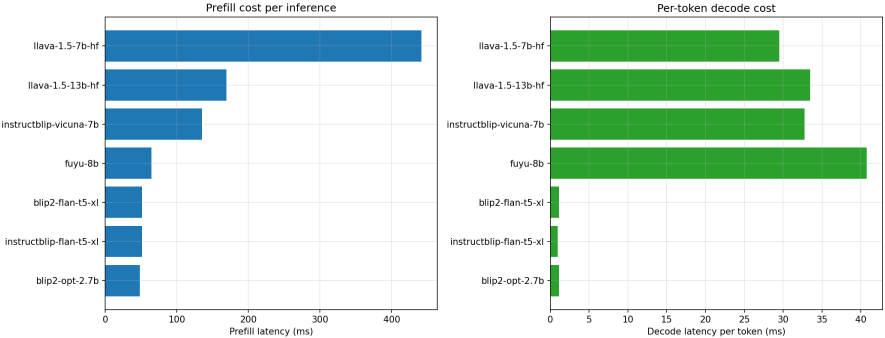
`instructblip-vicuna-7b`: great on COCO (0.96) but breaks on ScienceQA (11.18) — likely an instruction-tuning gap on science-style prompts.

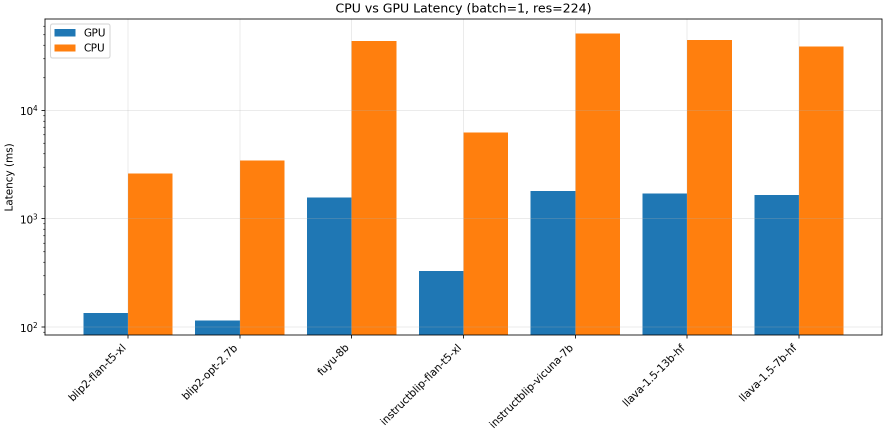
# Backup: parameter breakdown




Language decoder dominates parameter count. Vision encoder $< 400$ M. Matches the runtime breakdown.

# Backup: prefill vs. decode (absolute)




`llava-1.5-13b` is in FP16 (does not fit in 48 GiB FP32). Re-running 7B in FP16 confirms the dtype flips the ordering.

# Backup: CPU vs GPU